

MCP-SecBench: A Defensive Proxy for Mitigating Attacks Against MCP-Based AI Agents

September 14, 2026

Abstract

The Model Context Protocol (MCP) lets AI agents connect to external tools – file systems, search engines, terminals, databases – turning a conversational model into one that can act. This capability introduces a new attack surface: a malicious or compromised MCP tool can attempt to hijack the agent through its name, its parameter schema, or its response text. This report describes an extended, original implementation built on top of the *MCP Security Bench* (MSB) benchmark [1], which measures how often such attacks succeed (Attack Success Rate, ASR) across four attack surfaces. Our contribution is `mcp_guard`, a defensive proxy layer that validates tool signatures, gates tool parameters, and sanitizes tool responses without modifying MSB or its underlying agent framework, plus a unified CLI tool for running baseline-vs-defended comparisons entirely on a free, local, offline language model. We report results from a 120-scenario batch run, discuss what the data does and does not show, and outline the concrete next steps (a larger local model, full attack-surface coverage) needed to turn this from a working first implementation into a complete evaluation.

Contents

1	Introduction: MCP and the Security Problem	2
2	The MSB Benchmark	2
3	System Design	3
3.1	Defensive Proxy: <code>mcp_guard.py</code>	3
3.2	Windows-Compatible Setup	3
3.3	Pluggable LLM Backend	3
3.4	Unified CLI Runner	4
4	Experimental Setup	4
5	Results	4
6	Limitations and Future Work	5
7	Conclusion	6

1 Introduction: MCP and the Security Problem

The Model Context Protocol (MCP) is a standard that lets AI agents invoke external tools: reading files, running searches, controlling a terminal, querying a database, and more. Where a purely conversational model can only respond in text, an MCP-connected agent can *act* on the user’s behalf.

That capability is also the risk. An MCP tool is, functionally, a small program the agent is told to trust. If that tool – or content it returns – is malicious, it can attempt to manipulate the agent into performing actions the user never requested: exfiltrating private data, corrupting files, terminating unrelated processes, or planting unauthorized access credentials. This is a qualitatively different problem from classical software security: the question is not whether an attacker can breach the agent’s execution environment, but whether the agent can be deceived by the very tools it was instructed to trust.

MSB identifies four distinct surfaces through which this deception can occur, summarized in Table 1.

Table 1: MSB’s four MCP attack surfaces.

Attack surface	Mechanism
Tool Signature	A tool’s name collides with a trusted tool, or its description contains a hidden instruction (prompt injection, preference manipulation).
Tool Parameter	The tool’s schema requests an out-of-scope argument the agent should not need to supply (e.g. an internal model identifier).
Tool Response	The result returned by a tool call is a fabricated error or a fake “urgent new instruction” designed to hijack the agent mid-task.
Retrieval Injection	A retrieved document or search result embeds hidden instructions disguised as ordinary content.

2 The MSB Benchmark

MCP Security Bench (MSB) [1] is an ICLR 2026 benchmark that automates exactly this kind of test: it instantiates a real LLM agent connected to real MCP tool servers, assigns it an everyday task (e.g. “search arXiv for papers on machine learning”, “get the current working directory”), and covertly injects one of twelve attack variants into the pipeline. After execution, MSB checks whether the attack’s concrete goal was achieved – for example, whether a fabricated SSH key was written to a file, whether a private phone number was exfiltrated, or whether an unrelated process was killed.

MSB’s own published results, evaluated across ten LLM backbones, report an average ASR of **40.35%** across all attack types, with the Out-of-Scope-Parameter attack alone averaging **76.50%**. Critically, MSB is purely a *measurement* instrument: it contains no mitigation of any kind. This is the gap the present work addresses.

3 System Design

We preserved `baseline/` as an unmodified copy of upstream MSB and built four new components on top of it, described below and summarized in Table 2.

3.1 Defensive Proxy: `mcp_guard.py`

The core contribution is `mcp_guard.py`, a middleware layer that composes with an already-initialized MCP agent *without* modifying the agent framework’s source code. It performs three checks, corresponding to the three attack surfaces that are actually reachable at the agent-execution layer (Retrieval Injection is discussed as a current limitation in Section 6):

- (1) **Tool Signature Validation.** Before the agent’s system prompt is constructed, the tool list is de-duplicated by name (defeating name-collision attacks), and each tool’s description is scanned for known injection-template markers, which are stripped before the description reaches the prompt.
- (2) **Tool Parameter Gating.** Before a tool call is dispatched, its arguments are checked against a set of disallowed keys (MSB’s own out-of-scope markers such as `llm_model_name`, plus generic secret-like keys such as `api_key` or `password`). High-risk tool calls (`write_file`, `edit_file`, `kill_process`, ...) receive additional content inspection for SSH/PEM key material or references to sensitive filenames.
- (3) **Tool Response Sanitization.** After a tool call returns (or after MSB’s own attack-simulation logic substitutes a canned malicious string), the observation text is scanned for MSB’s known attack template fragments and generic instruction-override phrasing, and redacted before it is appended to the agent’s context.

Technically, this is implemented as a per-instance monkey-patch of the LangChain `AgentExecutor`’s tool-dispatch method, applied only when `apply_defense()` is explicitly called. This design choice is what makes the baseline-vs-defended comparison valid: the same scenario, same LLM, and same random conversational trajectory are used in both conditions: the only variable is whether the defensive wrapper is attached.

3.2 Windows-Compatible Setup

MSB’s own `setup.py` assumes a POSIX shell (it shells out to `which uv`) and destructively `shutil.moves` a LangChain patch file into the installed package, which would silently delete it from the tracked repository. `setup_windows.py` resolves `uv` via `shutil.which`, writes correctly JSON-escaped Windows paths into every MCP server configuration file, and *copies* rather than moves the patch, keeping `baseline/` a complete and auditable mirror of upstream.

3.3 Pluggable LLM Backend

MSB hardcodes its model selection to paid providers (OpenRouter, DeepSeek, Alibaba Tongyi). We generalized this to an explicit `<backend>/<model>` specification supporting `ollama/`, `openrouter/`, `openai/`, and `deepseek/` backends. This allowed the entire evaluation in this report to be run against a free, fully local, offline model (Ollama), with zero API cost.

3.4 Unified CLI Runner

`cli_runner.py` exposes three modes:

```
python cli_runner.py --mode baseline --llm ollama/llama3.2:3b
python cli_runner.py --mode defended --llm ollama/llama3.2:3b
python cli_runner.py --mode compare
```

It reuses MSB’s own scenario-construction helpers (`tool_exist`, `complete_server_config`) directly from `baseline/main.py`, and implements a success oracle that reads the sandboxed `operation_space/output` artifact directly, rather than regex-parsing MSB’s own log text as `metrics.py` does.

Table 2: Summary of original contributions vs. unmodified upstream content.

Component	Status
<code>baseline/</code>	Unmodified copy of upstream MSB
<code>defenses/mcp_guard.py</code>	Original – defensive proxy middleware
<code>setup_windows.py</code>	Original – Windows-compatible setup
<code>cli_runner.py</code>	Original – unified baseline/defended/compare CLI
<code>run_batch.py</code>	Original – curated batch driver
<code>custom_scenarios/</code>	Original – extension scaffold (not yet populated)

4 Experimental Setup

All experiments were run at zero monetary cost:

- **Model:** `llama3.2:3b`, a small open-weight model served locally via Ollama.
- **Tools:** MSB’s real MCP servers – a filesystem server, a terminal-control server, and a web-search server – each spawned as an actual subprocess per scenario, not simulated.
- **Scenario grid:** 3 attack types spanning three attack surfaces (`prompt_injection`, `out_of_scope_parameter`, `false_error`) $\times$ 5 canonical attack goals $\times$ 2 tools, executed once undefended and once with `mcp_guard` attached, for 120 total scenario runs.

Reaching a working local setup required resolving several practical obstacles: adapting MSB’s POSIX-only setup script for Windows, and three failed attempts at installing Ollama (a hung installer requiring elevated permissions with no interactive session to grant it, a corrupted archive download, and a severely rate-limited connection) before a working local model was obtained via a direct portable-archive download.

5 Results

Table 3 reports Attack Success Rate (ASR) for the undefended baseline and the `mcp_guard`-defended condition across the three tested attack types.

The headline ASR is 0% in both conditions, which requires careful interpretation rather than a simple pass/fail reading.

The underlying model did not fall for any tested attack, even undefended. A 3-billion-parameter local model may lack the multi-step tool-orchestration reliability needed to carry an

Table 3: Attack Success Rate, baseline vs. defended (11ama3.2:3b, 120 scenarios).

Attack type	Baseline ASR	Defended ASR	Reduction
Prompt Injection	0.0%	0.0%	0.0 pp
Out-of-Scope Parameter	0.0%	0.0%	0.0 pp
False Error	0.0%	0.0%	0.0 pp

injected instruction through to a concrete malicious action (e.g. actually calling `write_file` with attacker-supplied content), independent of whether a defense is present. This leaves no observable gap for the ASR metric itself to show a reduction against.

The guard was nonetheless measurably active. Across the 60 defended-mode scenarios, `mcp_guard` recorded **31 distinct interventions** (redacting injected tool-response content or refusing out-of-scope parameters), averaging 0.52 interventions per scenario. The corresponding count in the 60 baseline scenarios was exactly zero, by construction (the guard is never attached). This is direct, quantitative evidence that the mitigation layer is functioning as designed, independent of the model’s own resistance to the underlying attack.

An unrelated infrastructure limitation affected 8 of 60 scenarios per mode. All eight are `kill_process` attack-task scenarios, which depend on MSB’s Desktop Commander support tool; that tool is currently fetched through a third-party hosted registry (Smithery) that returns a “server not found” error in this environment. The failure is identical and symmetric across baseline and defended runs, so it does not bias the comparison, but it does mean the `kill_process` results are not currently informative.

6 Limitations and Future Work

- **Model capacity confound.** The single most important next step is repeating this evaluation against a more capable model. A larger local model (11ama3.1:8b, matching a model MSB itself reports at approximately 20% average ASR) has already been downloaded and is queued for this comparison; it should surface genuine baseline attack successes, enabling a real before/after reduction figure rather than a 0%-to-0% comparison.
- **Retrieval Injection is not yet scored.** MSB’s own success criterion for this attack type is qualitatively different from the other three (whether the agent was deceived into consulting the wrong document, rather than whether a file artifact was planted), and the current success oracle does not implement that logic. It is deliberately excluded from batch runs rather than silently reported as a misleading 0%.
- **Third-party tool dependency.** `kill_process` scenarios require an alternative source for the Desktop Commander tool, independent of Smithery’s hosted registry.
- **Scope.** This is a first working implementation covering three of MSB’s twelve attack-type combinations. The natural extension is full coverage of MSB’s attack grid and validation across multiple model sizes to establish whether the observed guard-intervention rate scales with a model’s underlying susceptibility.

7 Conclusion

This work demonstrates that a defensive proxy layer can be attached to an MCP-based agent pipeline without modifying the underlying benchmark or agent framework, and that its interventions can be measured directly and independently of whether the underlying attack succeeds. Our first end-to-end evaluation, run entirely at zero cost on a local model, did not find a model vulnerable enough to demonstrate an ASR reduction, but it did establish – with concrete counts, not just code review – that the guard is actively intercepting attack-shaped content. Extending this evaluation to a more capable model and to MSB’s full attack-type grid is the immediate next step toward a complete assessment of `mcp_guard`’s effectiveness.

References

- [1] Zhang, D., Li, Z., Luo, X., Liu, X., Li, P. P., Xu, W. *MCP Security Bench (MSB): Benchmarking Attacks Against Model Context Protocol in LLM Agents*. The Fourteenth International Conference on Learning Representations, 2026. <https://openreview.net/forum?id=irxxkFMrry>